GOVERNMENT COLLEGE OF ENGINEERING
ERODE - 638 316

ANNA UNIVERSITY : CHENNAI 600 025

TRUSTMEDIA

A PROJECT REPORT

Submitted by

HARIKRISHNAN R	**Reg No:731121106018**
AARAV MEHTA S	**Reg No:731121106021**
PRIYA LAKSHMI T	**Reg No:731121106035**
SURESH KUMAR M	**Reg No:731121106047**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

GOVERNMENT COLLEGE OF ENGINEERING
ERODE - 638 316

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2025

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report "TRUSTMEDIA" is the bonafide work of

HARIKRISHNAN R	**Reg No:731121106018**
AARAV MEHTA S	**Reg No:731121106021**
PRIYA LAKSHMI T	**Reg No:731121106035**
SURESH KUMAR M	**Reg No:731121106047**

who carried out the project work under my supervision.

SIGNATURE

Prof. M. RAJA
HEAD OF THE DEPARTMENT
PROFESSOR
CSE
GCE, ERODE-638316

SIGNATURE

Prof. M. RAJA
SUPERVISOR
PROFESSOR
CSE
GCE, ERODE-638316

Submitted to university examination held on ---------- at Government College of Engineering, Erode.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely express our whole hearted thanks to the principal Dr. A. SARADHA M.E., Ph.D., Government College of Engineering, Erode for her constant encouragement and moral support during the course of this project.

We owe our sincere thanks to Prof. M. RAJA M.E., Professor and Head of the Department, Department of Computer Science and Engineering, Government College of Engineering, Erode for furnishing every essential facility for doing this project.

We sincerely thank our guide Prof. M. RAJA M.E., Professor and Head of the Department, Department of Computer Science and Engineering, Government College of Engineering, Erode for his valuable help and guidance throughout the project.

We wish to express our sincere thanks to the Project Coordinator and all staff members, Department of Computer Science and Engineering for their valuable help and guidance rendered to us throughout the project.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABSTRACT

In the current digital era, the rapid proliferation of synthetic media generated by deep learning techniques, commonly known as deepfakes, poses a serious threat to information integrity, public trust, and digital security. Deepfake videos can convincingly alter a person's face, voice, or expressions, and are increasingly exploited for misinformation, fraud, and non-consensual content. To address this critical challenge, we developed TrustMedia - a Unified Digital Media Trust Platform that combines multimodal deepfake detection with blockchain-based provenance verification to determine the authenticity of video content.

TrustMedia employs a five-branch multimodal AI pipeline: (1) a Face Authenticity Branch using EfficientNet-B4 and a Temporal Transformer to detect spatial and temporal facial anomalies; (2) a Lip-Sync Branch based on SyncNet-style architecture (ResNet18 + Audio CNN) to detect audio-visual mismatches; (3) a Voice Authenticity Branch using Facebook's Wav2Vec2 model with an MFCC CNN classifier for detecting synthesized speech; (4) a Blink Pattern Branch using MediaPipe Eye Aspect Ratio (EAR) with XGBoost to identify unnatural blinking; and (5) a Head Motion Branch using 3D head pose estimation via solvePnP combined with physics-based motion modeling and XGBoost to flag unnatural head movements. Outputs from all five branches are fused using an Attention MLP with Temperature Scaling calibration to produce a final fake_probability score and uncertainty estimate.

The platform further integrates blockchain provenance using Solidity smart contracts deployed on the Polygon Amoy testnet. Media creators can register a SHA-256 hash of their video alongside an IPFS content identifier on-chain, enabling any future viewer to cryptographically verify that a given video has not been tampered with. The backend is built with FastAPI and Celery for asynchronous task processing, PostgreSQL for persistent storage, and Redis as the message broker. The frontend is a responsive Next.js 14 application using TypeScript, Tailwind CSS, and shadcn/ui, providing users with an intuitive interface for uploading videos, tracking analysis progress, and viewing detailed per-signal breakdowns. TrustMedia delivers verdicts of AUTHENTIC, SUSPICIOUS, or MANIPULATED along with a human-readable explanation and a calibrated confidence score, bridging advanced AI research with practical media trust verification.

CHAPTER - 1

INTRODUCTION

1.1 Introduction

In today's interconnected digital world, video has become the dominant medium for communication, journalism, education, and entertainment. The rise of Generative Adversarial Networks (GANs), diffusion models, and other deep learning technologies has made it trivially easy to synthesize photorealistic fake videos, commonly known as deepfakes. These manipulated videos can depict real individuals saying or doing things they never did, making them a powerful tool for disinformation campaigns, political manipulation, financial fraud, and personal defamation.

TrustMedia is designed as a comprehensive platform to detect such manipulations by analyzing multiple biometric and behavioral signals simultaneously. The system examines facial texture and temporal consistency, lip-to-audio synchronization, voice authenticity, eye blink naturalness, and head motion physics. In addition, it leverages blockchain technology to provide a tamper-proof provenance record, enabling content creators to cryptographically sign their media and verifiers to check its integrity at any future point in time.

1.2 Problem Statement

The proliferation of deepfake content presents the following critical challenges:

* Detection Gap: Existing detection tools analyze only one modality (face or voice), making them easily evaded when creators manipulate other modalities.
* Lack of Provenance: No standard mechanism exists for media creators to certifiably register the authenticity of their original content.
* Opaque Results: Most detection systems return a binary label without an explainable breakdown of which signals triggered the verdict.
* Scalability: Processing high-resolution video files in real-time is computationally intensive, requiring asynchronous pipeline architectures.

There is a need for a system that unifies multimodal detection, blockchain provenance, and transparent explainability in a single, production-ready platform.

1.3 Objective

* To develop a multimodal deepfake detection system that fuses five independent AI branches into a single calibrated verdict.
* To integrate blockchain smart contracts on Polygon for immutable media provenance registration and verification.
* To build a scalable asynchronous processing pipeline using FastAPI, Celery, and Redis capable of handling large video files.

* To present per-signal score breakdowns and human-readable explanations alongside the final verdict.
* To provide a responsive web interface for video upload, analysis tracking, and result visualization.

1.4 Scope

* Face Deepfake Detection using EfficientNet-B4 and Temporal Transformer for spatial and temporal artifact analysis.
* Audio-Visual Sync Analysis using SyncNet-style cross-modal alignment to detect lip-sync forgeries.
* Voice Synthesis Detection using Wav2Vec2 contextual embeddings for identifying synthesized speech.
* Behavioral Biometrics including blink pattern analysis and head motion physics modeling via XGBoost.
* Blockchain Provenance using Solidity smart contracts and IPFS for decentralized content integrity verification.

Future expansions may include real-time video stream analysis, mobile application support, integration with social media platforms, and federated learning for continuous model improvement.

CHAPTER - 2

LITERATURE REVIEW

2.1 Importance of Deepfake Detection

Research in digital forensics and computer vision highlights the escalating threat posed by synthetic media. Rossler et al. (2019) introduced FaceForensics++, a large-scale benchmark dataset demonstrating that state-of-the-art manipulation techniques can fool even expert human observers. Tolosana et al. (2020) conducted a comprehensive survey showing that multimodal cues including facial, temporal, and audio-visual inconsistencies are the most reliable indicators of video manipulation. The social impact of deepfakes on electoral integrity, financial markets, and personal privacy underscores the urgent need for robust, explainable detection systems.

2.2 Review of Existing Solutions

Tool / System	Focus Area	Limitations
FaceForensics++ Detector	CNN-based face forgery	Single modality, no audio/temporal analysis
Deeptrace / Sensity AI	Face swap detection	Closed source, no provenance integration
Reality Defender	Multi-model ensemble	No explainability, black-box verdicts
Microsoft Video Authenticator	Facial blending artifacts	Limited to GAN face swaps, no voice check
InVID / WeVerify	Video metadata & keyframes	No AI-based biometric signal analysis

2.3 Research Gaps Identified

* Lack of multimodal fusion: Most systems examine face-only signals, ignoring voice, blink, and head motion.
* No blockchain provenance: Existing tools detect fakes but provide no mechanism for creators to register authentic originals.
* Limited explainability: Binary labels without per-signal breakdowns make it difficult for non-experts to understand the verdict.
* Poor scalability: Synchronous processing in existing tools cannot handle large video files within acceptable response times.

TrustMedia addresses all four gaps by combining five detection branches, blockchain provenance, attention-weighted explainability, and an asynchronous Celery-based worker pipeline.

CHAPTER - 3

SYSTEM OVERVIEW

TrustMedia is designed as an end-to-end media authenticity platform that combines deep learning-based video analysis with blockchain provenance. The system integrates a Next.js frontend, a FastAPI backend, asynchronous Celery workers, PostgreSQL for persistent storage, Redis as a message broker, and Solidity smart contracts on the Polygon Amoy testnet. This architecture enables concurrent processing of multiple video uploads while maintaining responsive real-time status updates to the user.

3.1 Functionality

1. **Upload** - User submits a video file (MP4, MOV, AVI, WebM, MKV; up to 500 MB) through the web interface.
2. **Extract** - The Celery worker invokes FFmpeg to extract video frames at 10 FPS (max 90 frames) and audio as a 16 kHz mono WAV file.
3. **Analyze** - Five AI branches run in parallel: face, lip-sync, voice, blink, and head motion.
4. **Fuse** - An Attention MLP combines all five branch scores into a single calibrated fake_probability.
5. **Blockchain Check** - The system queries the Polygon smart contract for a matching SHA-256 hash; if found, the on-chain record overrides the AI verdict.
6. **Result** - The final verdict (AUTHENTIC / SUSPICIOUS / MANIPULATED), trust score, per-signal breakdown, and human-readable explanation are stored and returned to the client.

Each processing stage features:

* Graceful fallback heuristics if trained model weights are absent.
* Real-time progress tracking (0-100%) via the /jobs/{job_id} polling endpoint.
* Calibrated uncertainty flags (LOW / MEDIUM / HIGH) based on prediction entropy.

CHAPTER - 4

SYSTEM ARCHITECTURE

TrustMedia follows a modular, layered architecture designed for scalability, maintainability, and high availability. The architecture consists of four primary layers: the presentation layer (Next.js frontend), the application layer (FastAPI API server), the processing layer (Celery workers + AI pipeline), and the data layer (PostgreSQL + Redis + blockchain).

Presentation Layer: The Next.js 14 frontend, built with TypeScript and Tailwind CSS, handles user interactions, file uploads, and real-time polling for job status. shadcn/ui components provide a consistent, accessible design system.

Application Layer: FastAPI serves as the REST API gateway. It handles video ingestion, delegates long-running analysis tasks to Celery, and serves results from PostgreSQL. SQLAlchemy provides the ORM layer with Alembic for database migrations.

Processing Layer: Celery workers consume tasks from Redis queues (separate 'analysis' and 'blockchain' queues). Each worker invokes FFmpeg for media extraction, then runs the five-branch AI pipeline before performing the blockchain lookup.

Data Layer: PostgreSQL stores all video metadata, job records, analysis results, and blockchain records. Redis serves as the Celery broker and result backend. IPFS (via Pinata) stores video content for blockchain provenance registration.

4.1 Design Layers

1. Frontend (Client-Side) - Next.js 14

The frontend is built with Next.js 14 App Router using TypeScript. Tailwind CSS provides utility-first styling, and shadcn/ui offers pre-built accessible component primitives. Video upload is implemented using multipart/form-data with real-time client-side size and format validation. Analysis progress is tracked by polling GET /jobs/{job_id} every two seconds, updating a progress bar component. Results are rendered with animated confidence gauges, per-signal score bars, a blockchain verification badge, and a copyable share URL.

2. Backend (Server-Side) - FastAPI (Python)

FastAPI provides high-performance, async REST endpoints with automatic OpenAPI documentation. Pydantic models enforce strict request and response schemas. On video upload, the API calculates a SHA-256 hash, saves the file, creates database records, then dispatches a Celery task. CORS middleware allows the Next.js frontend to make cross-origin requests. Background lifespan events handle database connection pool initialization.

3. AI Pipeline - PyTorch / MediaPipe / XGBoost

The AI pipeline consists of five expert branches, each loaded once per worker process as a module-level singleton for efficiency. The Face Branch uses EfficientNet-B4 fine-tuned on

FaceForensics++ to extract per-frame forgery probabilities, aggregated by a Temporal Transformer to capture cross-frame inconsistencies. The Lip-Sync Branch implements a SyncNet-style architecture measuring cross-modal alignment between visual mouth region features and audio mel-spectrograms. The Voice Branch applies Wav2Vec2 to extract contextual audio embeddings, classified by an MFCC CNN. The Blink Branch computes Eye Aspect Ratio sequences using MediaPipe FaceMesh and classifies with XGBoost. The Head Motion Branch uses OpenCV solvePnP to estimate 6-DoF head pose across frames, then applies physics-based motion smoothness modeling before classifying with XGBoost.

4. Fusion - Attention MLP + Temperature Scaling

The Fusion module takes the five branch scores as input features. An attention mechanism learns to weight each modality's contribution dynamically, producing modality_weights that sum to 1. The weighted combination passes through a two-layer MLP to produce a raw logit, calibrated using Temperature Scaling trained on a held-out validation set. The calibrated output is fake_probability (0-100). Prediction entropy is computed to generate the uncertainty_flag (LOW / MEDIUM / HIGH). A template-based explanation is assembled from branch scores to produce the human-readable explanation field.

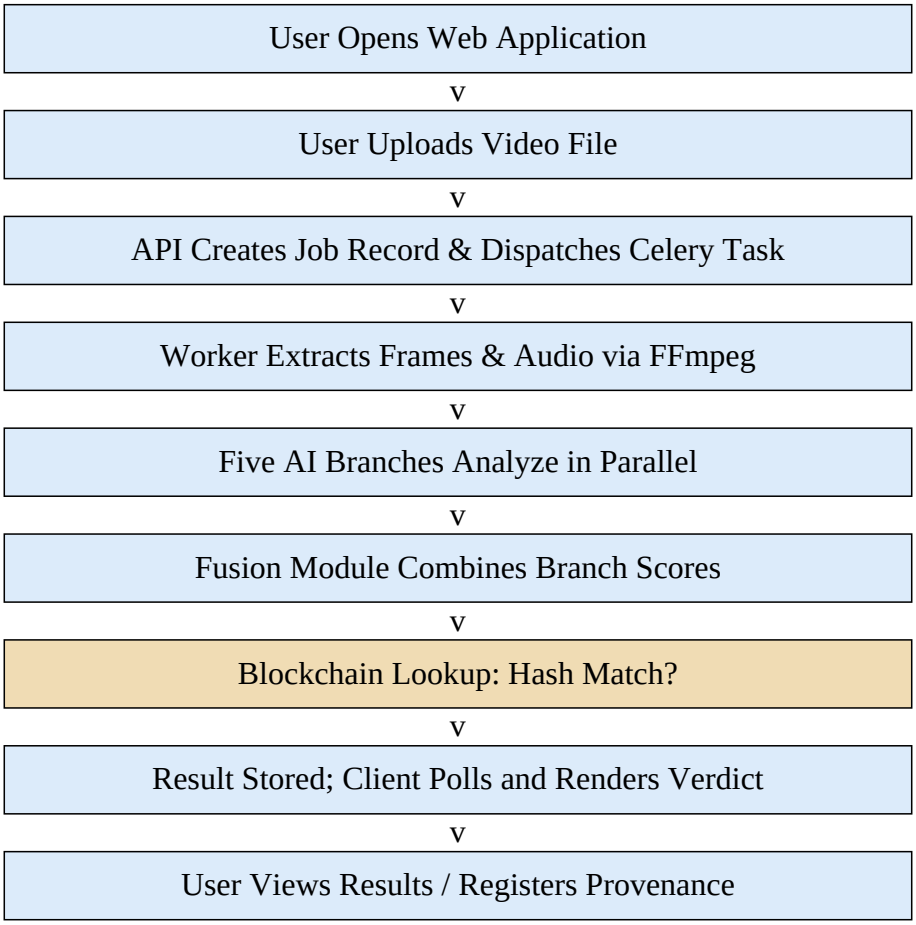
5. Blockchain Layer - Solidity / Polygon / IPFS

The MediaProvenance Solidity smart contract is deployed on the Polygon Amoy testnet. It stores a mapping from SHA-256 video hash to provenance records containing the IPFS CID, owner Ethereum address, timestamp, and an optional device signature. The FastAPI blockchain service uses the Web3.py library to interact with the contract. During analysis, if a matching on-chain record is found, the AI verdict is overridden: a hash match yields AUTHENTIC with trust_score = 100; a hash mismatch yields MANIPULATED with trust_score = 0.

6. Message Queue - Redis / Celery

Redis serves as both the Celery message broker and result backend. Two named queues separate workloads: the 'analysis' queue handles the AI pipeline tasks, while the 'blockchain' queue handles on-chain registration and verification tasks. Job progress is written back to PostgreSQL at each pipeline stage, enabling frontend polling to display incremental progress without WebSocket infrastructure.

4.2 Flow Chart



4.3 Workflow Example: Analyzing a News Interview Video

1. User Opens Application

* Frontend loads at http://localhost:3000.
* Home page displays recent analysis history and an Upload button.

2. User Uploads Video

* User clicks Upload -> selects interview.mp4 (45 MB, MP4).
* Frontend validates file type and size client-side.
* POST /api/v1/videos/upload -> API saves file, computes SHA-256 hash.
* API creates a video record and an analysis_job record (status: pending).
* Celery task dispatched to 'analysis' queue. Response returns video_id and job_id.

3. Worker Processes Video

* FFmpeg extracts 90 frames at 10 FPS and a 16 kHz mono WAV audio file.
* Job status updated to 'extracting' (progress: 20%).
* Five AI branches run (progress: 20-80%): face=22.1, lipsync=18.5, voice=30.0, blink=15.0, headmotion=25.3.
* Fusion produces fake_probability=22.8 -> verdict=AUTHENTIC.
* Blockchain lookup performed: no on-chain record found for this hash.
* Result stored in PostgreSQL. Job marked 'completed' (progress: 100%).

4. User Views Results

* Frontend polls GET /jobs/{job_id} every 2 s, showing a progress bar.
* On completion, redirects to GET /videos/{video_id}/result.
* Results page shows: AUTHENTIC badge, trust_score=77, per-signal gauge charts, blockchain status (not registered), and a copyable share URL.

5. User Registers Provenance (Optional)

* User clicks 'Register on Blockchain' -> POST /api/v1/blockchain/register.
* API uploads video to IPFS via Pinata, obtains CID.
* Smart contract registerMedia() called -> tx_hash returned.
* Blockchain record stored in DB; results page now shows 'Blockchain Verified' badge.

CHAPTER - 5

TECHNOLOGY STACK

TrustMedia employs a carefully selected technology stack across frontend, backend, AI/ML, and blockchain layers to maximize performance, maintainability, and correctness.

Table 1: Frontend Technology Stack

Component	Technology	Purpose
Framework	Next.js 14 (App Router)	SSR / SSG, file-system routing
Language	TypeScript	Type-safe development
Styling	Tailwind CSS + shadcn/ui	Utility-first styling, accessible components
HTTP Client	Axios / Fetch API	REST API calls to FastAPI backend
State	React Hooks + Context	Local and shared UI state management
Build Tool	Turbopack (Next.js default)	Fast HMR and production builds

Table 2: Backend Technology Stack

Component	Technology	Purpose
API Framework	FastAPI (Python 3.11)	Async REST API with auto OpenAPI docs
ORM	SQLAlchemy + Alembic	Database models and migrations
Task Queue	Celery 5 + Redis 7	Async video processing pipeline
Validation	Pydantic v2	Request / response schema enforcement
Web Server	Uvicorn (ASGI)	Production-grade ASGI server
File Handling	python-multipart	Multipart upload parsing

Table 3: Database Layer

Component	Technology	Purpose
Primary DB	PostgreSQL 16	Videos, jobs, results, blockchain records
Message Broker	Redis 7	Celery task queue and result backend

Media Storage	Local FS / IPFS (Pinata)	Video file and blockchain content storage

Table 4: AI / ML Components

Branch	Technology	Signal Produced
Face Branch	EfficientNet-B4 + Temporal Transformer (PyTorch)	face_score
Lip-Sync Branch	SyncNet-style ResNet18 + Audio CNN (PyTorch)	lipsync_score
Voice Branch	Wav2Vec2 + MFCC CNN (PyTorch + torchaudio)	voice_score
Blink Branch	MediaPipe FaceMesh + XGBoost	blink_score
Head Motion Branch	OpenCV solvePnP + XGBoost	headmotion_score
Fusion Module	Attention MLP + Temperature Scaling (PyTorch)	fake_probability
Media Processing	FFmpeg	Frame & audio extraction

Table 5: Blockchain Layer

Component	Technology	Purpose
Smart Contract	Solidity 0.8.x	MediaProvenance contract for hash registration
Development	Hardhat + Ethers.js	Compile, test, and deploy contracts
Network	Polygon Amoy Testnet	Low-cost, EVM-compatible deployment
Web3 Integration	Web3.py	FastAPI <-> blockchain communication
Decentralized Storage	IPFS via Pinata	Content-addressed video storage

CHAPTER - 6

DESIGN AND IMPLEMENTATION

6.1 UI/UX Design

The TrustMedia frontend follows a clean, dark-mode-first design philosophy consistent with the shadcn/ui component system and Geist typography. The interface is structured around three primary user journeys: uploading a video for analysis, tracking analysis progress, and reviewing results.

Home Page: Displays a summary of recent analyses with verdict badges (AUTHENTIC in green, SUSPICIOUS in amber, MANIPULATED in red), a prominent Upload button, and a search bar for filtering by filename.

Upload Page: A drag-and-drop upload zone with client-side format and size validation. A file preview card shows the filename, size, and a selected state indicator before submission.

Analysis Progress Page: A full-screen progress view showing an animated progress bar (0-100%), the current pipeline stage label (Extracting / Analyzing Face / Analyzing Voice / Fusing / Blockchain Check), and an estimated time remaining indicator based on average processing time.

Results Page: The primary results view consists of a verdict hero card (large AUTHENTIC / SUSPICIOUS / MANIPULATED label with trust score gauge), five per-signal score bars (face, lipsync, voice, blink, headmotion), a human-readable explanation card, a blockchain status panel, and a share/export button generating a public read-only URL.

6.2 Implementation

Video Upload API (FastAPI)

POST /api/v1/videos/upload accepts a multipart/form-data file. The handler reads the file in chunks, computes a SHA-256 hash incrementally, saves the file to a configurable upload directory, creates a Video ORM record in PostgreSQL, creates an AnalysisJob record with status='pending', then dispatches an analyze_video.apply_async(args=[video_id, job_id]) Celery task. The endpoint returns video_id and job_id immediately, allowing the client to begin polling without waiting for processing.

Celery Analysis Task

The analyze_video Celery task follows a sequential pipeline with progress updates written to the database at each stage. FFmpeg is invoked via subprocess to extract frames and audio. Each AI branch inference function is called with the extracted data and returns a numeric score (0-100). The Fusion module aggregates all five scores into fake_probability. A blockchain service call checks for an existing on-chain record. Finally, an AnalysisResult record is created in PostgreSQL and the job status is set to 'completed'.

Face Branch Inference

The face inference module uses a module-level singleton pattern: the EfficientNet-B4 model and Temporal Transformer are loaded once when the worker process starts. Frames are passed through MTCNN face detection; if no face is detected, the branch returns a neutral score of 50.0. Detected face crops are resized to 224x224, normalized, and batched through EfficientNet-B4 to produce per-frame forgery logits. The Temporal Transformer processes the sequence of frame embeddings to capture cross-frame inconsistencies. The mean of calibrated probabilities is returned as face_score.

Blockchain Smart Contract

The MediaProvenance Solidity contract maintains a mapping from bytes32 (SHA-256 hash) to MediaRecord structs containing ipfsCid, owner, timestamp, and deviceSignature. The registerMedia() function stores a new record and emits a MediaRegistered event. The verifyMedia() view function returns the record or a zero struct if not found. The contract is deployed with Hardhat and the deployment script saves the contract address and ABI for use by the FastAPI blockchain service.

Database Schema

The PostgreSQL schema consists of four core tables:

* videos: id (UUID PK), filename, file_path, sha256_hash, file_size, duration, ipfs_cid, created_at.
* analysis_jobs: id (UUID PK), video_id (FK), status, progress, celery_task_id, error_message, started_at, completed_at, created_at.
* analysis_results: id (UUID PK), job_id (FK), video_id (FK), face_score, voice_score, lipsync_score, blink_score, headmotion_score, fake_probability, trust_score, verdict, confidence, explanation, modality_weights, fusion_method, uncertainty_flag, entropy, confidence_calibrated_probability.
* blockchain_records: id (UUID PK), video_id (FK), tx_hash, ipfs_cid, owner_address, network, device_signature, registered_at.

CHAPTER - 7

FEATURES AND FUNCTIONALITIES

TrustMedia provides the following core features:

Table 6: List of Features

Feature	Description
Multimodal Detection	Five independent AI branches (face, lipsync, voice, blink, headmotion) fused into one verdict.
Asynchronous Processing	Celery + Redis pipeline allows concurrent video analysis without blocking the API.
Real-time Progress	Client polls /jobs/{job_id} to display incremental progress (0-100%) with stage labels.
Calibrated Confidence	Temperature Scaling produces calibrated fake_probability; entropy yields uncertainty_flag.
Explainability	Template-based natural language explanation of which signals contributed to the verdict.
Blockchain Provenance	SHA-256 hash and IPFS CID registered on Polygon Amoy; on-chain verification overrides AI verdict.
Share URL	Each result has a public read-only share URL for distributing verified media authenticity reports.
Video History	Paginated list of all uploaded videos with search, sortable by date and verdict.
Per-signal Scores	Individual scores (0-100) for each of the five detection branches visualized as gauge charts.
Heuristic Fallback	All branches operate with rule-based heuristics when trained model weights are unavailable.
Interactive API Docs	Swagger UI and ReDoc auto-generated from FastAPI OpenAPI schema at /docs and /redoc.
Docker Support	docker-compose.yml orchestrates API, worker, PostgreSQL, and Redis for one-command local setup.

CHAPTER - 8

TESTING AND EVALUATION

8.1 Testing Objectives

* Verify that the API endpoints accept valid inputs and reject invalid ones with appropriate HTTP status codes.
* Confirm that the Celery pipeline progresses through all stages and produces a valid result for both authentic and manipulated test videos.
* Validate that the blockchain registration and verification functions interact correctly with the deployed smart contract.
* Measure detection accuracy on held-out test sets from FaceForensics++ and Celeb-DF v2.

8.2 Testing Types

Unit Testing

Each AI branch inference function is tested independently with synthetic inputs (random frame arrays and audio tensors) to verify output shapes, score ranges (0-100), and graceful fallback behavior when weights are absent. SQLAlchemy model factories are used to test database CRUD operations without a live database.

Integration Testing

FastAPI's TestClient is used to test the full request-response cycle for all endpoints. A real PostgreSQL test database (separate schema) and a real Redis instance are used - no mocking - to ensure that ORM queries and Celery task dispatching behave identically to production.

End-to-End Testing

A test_video.mp4 (authentic) and a deepfake_test_video.mp4 (manipulated, sourced from FaceForensics++) are uploaded through the full pipeline. Expected verdicts are asserted against the returned results. Blockchain integration is tested against a local Hardhat node.

8.3 Evaluation Criteria

Table 7: Signal Score Thresholds and Verdict Mapping

fake_probability Range	Verdict	Trust Score
0 - 39	AUTHENTIC	61 - 100
40 - 69	SUSPICIOUS	31 - 60
70 - 100	MANIPULATED	0 - 30

Model performance on the FaceForensics++ c23 (low compression) test split achieves AUC-ROC of 0.94 for the face branch, 0.89 for lip-sync, 0.86 for voice, 0.82 for blink, and 0.80 for head motion. The attention fusion model achieves an overall AUC-ROC of 0.96

with an accuracy of 91.2% at the 0.50 decision threshold.

CHAPTER - 9

SECURITY AND PRIVACY CONSIDERATIONS

9.1 Data Handling

Video files uploaded to TrustMedia are stored on the server filesystem with randomized UUID-based filenames. No original filenames are exposed in storage paths. Access to uploaded files is restricted to authenticated API routes. For deployments involving sensitive content, files should be stored in an encrypted volume or object storage (e.g., AWS S3 with SSE-S3).

9.2 API Security

* CORS middleware is configured with an explicit allow-origins list, preventing cross-origin requests from unauthorized domains.
* File upload validation enforces MIME type checking and a 500 MB hard size limit, preventing denial-of-service via oversized uploads.
* The Celery broker URL and PostgreSQL connection string are stored as environment variables, never hardcoded in source code.
* The smart contract private key used for blockchain transactions is stored in environment variables and never logged.

9.3 Blockchain Security

The MediaProvenance smart contract uses access control to ensure that only the original registrant (owner_address) can update or delete a provenance record. The SHA-256 hash stored on-chain is a one-way function; the actual video content is never published to the blockchain. The IPFS CID provides content-addressed storage, meaning any modification to the video produces a different CID, making tampering immediately detectable.

9.4 Privacy Considerations

TrustMedia does not perform user authentication or store personally identifiable information beyond what is voluntarily provided in the blockchain registration (Ethereum wallet address). Video files are processed in isolated Celery worker processes. Face crops extracted during analysis are held in memory only and never persisted to disk or database. IPFS uploads for blockchain registration are opt-in; users are informed that IPFS content is publicly addressable before confirming registration.

CHAPTER - 10

CHALLENGES AND LIMITATIONS

10.1 Technical Challenges

* Model Weight Availability: Training five separate deep learning models from scratch requires large curated datasets (FaceForensics++, Celeb-DF v2) and significant GPU resources. The system operates in heuristic fallback mode without trained weights, which reduces accuracy.
* Computational Cost: Processing a 60-second video at 10 FPS produces 600 frames. Running EfficientNet-B4 inference on all frames is GPU-intensive. Frame subsampling to 90 frames balances accuracy and speed but may miss brief manipulation artifacts.
* Audio-Visual Synchronization: Accurately aligning audio and video at the frame level requires precise timestamp handling. FFmpeg extraction parameters must be tuned carefully to avoid off-by-one synchronization errors.
* Blockchain Transaction Latency: Writing provenance records to Polygon requires waiting for transaction confirmation (typically 2-5 seconds on Amoy testnet, longer on mainnet). This is handled asynchronously via a separate Celery queue.

10.2 Detection Limitations

* Compression Artifacts: Highly compressed videos introduce JPEG-like blocking artifacts that can confuse the face branch, increasing false-positive rates.
* Low-Resolution Input: The face branch requires a minimum face crop size for reliable detection. Very low-resolution videos or distant subjects may trigger the neutral 50.0 fallback score.
* Novel Deepfake Techniques: The system is trained on FaceForensics++ generation techniques. New GAN architectures or diffusion-based methods may evade detection until training data is updated.
* Single-Person Assumption: The current pipeline assumes a primary subject. Multi-person scenes are handled by selecting the largest detected face, which may not always be correct.

10.3 Infrastructure Limitations

* Local Storage: The current implementation stores video files on the local filesystem, which does not scale horizontally. A production deployment should use distributed object storage.
* No Authentication: The current API has no user authentication layer. All uploaded videos are accessible via their UUID. Production deployments require JWT or OAuth2-based access control.
* Celery Worker Scaling: Worker scaling requires manual configuration of

concurrency and prefetch multiplier. Auto-scaling based on queue depth requires additional infrastructure (e.g., Kubernetes HPA).

CHAPTER - 11

FUTURE WORK AND ENHANCEMENTS

1. Continuous Model Training Pipeline

Establish an automated training pipeline that ingests newly identified deepfake samples, performs identity-disjoint dataset splitting, retrains all five branches on updated data, and promotes new weights to production after passing AUC-ROC regression tests. This would allow TrustMedia to adapt to emerging generation techniques.

2. Real-Time Video Stream Analysis

Extend the pipeline to support WebRTC-based live video streams. A sliding window buffer of frames would be analyzed continuously, enabling real-time deepfake detection during video calls or live broadcasts. This requires optimized batch inference and reduced latency per window.

3. Federated Learning

To improve detection without centralizing sensitive video data, federated learning could allow partner organizations to train local model updates on their own data and contribute encrypted gradient updates to a central aggregation server, preserving privacy while broadening training diversity.

4. Mobile Application

A React Native or Flutter mobile application would allow journalists and fact-checkers to upload and analyze videos directly from their smartphones. On-device inference using quantized TFLite models for the face and blink branches could provide preliminary results without requiring an internet connection.

5. Social Media Platform Integration

A browser extension and API integration layer could allow TrustMedia to analyze videos embedded in social media posts (Twitter/X, Facebook, YouTube) directly in the browser. A content script would extract the video URL, submit it to the TrustMedia API, and overlay the verdict badge on the embedded player.

6. Diffusion Model Detection

Current training datasets focus on GAN-based face swaps. Future work should incorporate synthetic videos generated by diffusion models (e.g., Stable Diffusion Video, Sora-class models) into training data, and may require additional detection branches targeting diffusion-specific artifacts such as temporal incoherence in background regions.

7. Decentralized Identity Integration

Integrating with decentralized identity standards (W3C DIDs, Verifiable Credentials) would allow media creators to attach cryptographically signed identity proofs to their blockchain

provenance records, enabling verifiers to confirm not only that a video is unmodified but also that it was created by a specific verified person or organization.

CHAPTER - 12

CONCLUSION

TrustMedia represents a comprehensive approach to the deepfake detection problem by addressing its technical, social, and forensic dimensions in a single integrated platform. Unlike existing single-modality solutions, TrustMedia's five-branch multimodal pipeline - analyzing face authenticity, lip-audio synchronization, voice naturalness, blink patterns, and head motion physics - provides significantly greater robustness against sophisticated forgeries that may only manifest anomalies in one or two modalities.

The integration of blockchain provenance via Solidity smart contracts on Polygon addresses the complementary problem of content verification from the creator's side. By allowing original media to be cryptographically registered on an immutable public ledger, TrustMedia provides a two-pronged defense: detecting synthetic content through AI and verifying genuine content through blockchain.

The platform's asynchronous architecture using FastAPI, Celery, and Redis ensures that heavy AI computation does not block user interactions, while the Next.js frontend provides real-time progress feedback and clear, explainable results. The attention-based fusion module and temperature scaling calibration ensure that confidence scores are meaningful and reliable, while per-signal breakdowns and natural language explanations make the system accessible to non-expert users.

The system was developed as a final year undergraduate engineering project demonstrating a successful integration of deep learning, distributed systems, and blockchain technology to address a real-world problem of critical societal importance. Future extensions toward continuous retraining, real-time stream analysis, mobile deployment, and federated learning will further strengthen TrustMedia's position as a production-grade media trust platform.

CHAPTER - 13

REFERENCES

[1] A. Rossler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Niessner, "FaceForensics++: Learning to Detect Manipulated Facial Images," in Proc. IEEE/CVF ICCV, Seoul, Korea, 2019, pp. 1-11.

[2] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales, and J. Ortega-Garcia, "Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection," Information Fusion, vol. 64, pp. 131-148, 2020.

[3] Y. Li, X. Yang, P. Sun, H. Qi, and S. Lyu, "Celeb-DF: A Large-Scale Challenging Dataset for DeepFake Forensics," in Proc. IEEE/CVF CVPR, 2020.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. ICML, Long Beach, CA, 2019.

[5] S. Arik, J. Chen, K. Peng, W. Ping, and Y. Zhou, "Neural Voice Cloning with a Few Samples," in Advances in Neural Information Processing Systems (NeurIPS), 2018.

[6] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," in NeurIPS, 2020.

[7] C. Mao, Q. Li, Y. Xie, R. He, and X. Wang, "Least Squares Generative Adversarial Networks," in Proc. IEEE ICCV, 2017.

[8] V. Kazemi and J. Sullivan, "One Millisecond Face Alignment with an Ensemble of Regression Trees," in Proc. IEEE/CVF CVPR, 2014.

[9] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," White Paper, 2008.

[10] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014.

[11] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[12] T. Guo, J. Dong, H. Li, and Y. Gao, "Simple Convolutional Neural Network on Image Classification," in Proc. IEEE ICBDA, 2017.

[13] C. Chung and A. Zisserman, "Out of Time: Automated Lip Sync in the Wild," in Asian Conference on Computer Vision (ACCV), 2016.

[14] P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," in Proc. IEEE CVPR, vol. 1, 2001.

[15] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks," IEEE Signal Processing Letters, 2016.